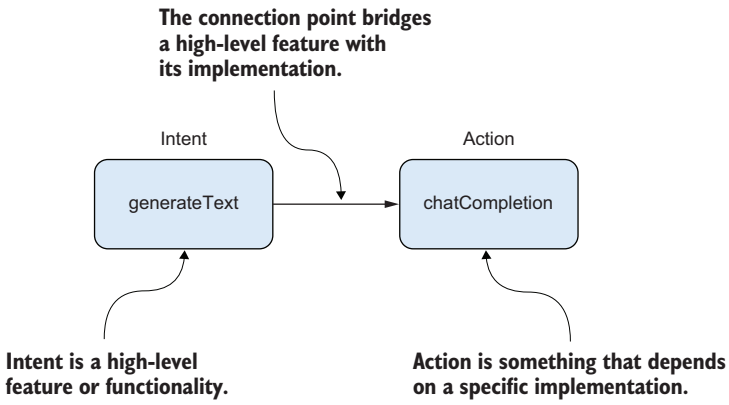


Figure 3.1 Separation of concerns between intents and actions. The intent (left) represents the high-level feature or functionality, such as `generateText`. The action (right) represents the specific implementation or concrete steps to fulfill the intent, such as making a `chatCompletion` request. The arrow signifies the connection point that bridges the intent with its corresponding action, facilitating communication and interaction between the two components.

To implement this separation in your code, you would typically define interfaces or abstractions for intents and actions. The connection point would be responsible for mapping the intents to their respective actions. This can be achieved through various mechanisms such as dependency injection, factory patterns, or configuration files.

Abstraction layers

Introduce abstraction layers between your application logic and external dependencies, such as AI providers or state management solutions. This will allow you to swap out these dependencies with minimal effect on your application's core functionality. We can see that in our existing chat application, we directly call the OpenAI client inside the route handler. This could be problematic if we decide to use a different provider in the future.

Incremental integration

Instead of a complete overhaul, I will introduce the new features one at a time. This reduces the risk of breaking existing functionality and makes the migration process more manageable.

Testing and validation

We will continue to validate our changes with each step to ensure nothing is broken, building a foundation that is both robust and flexible.

Documentation and knowledge sharing

Document your integration process, decisions, and any challenges you encounter. Share this knowledge with your team to ensure a smooth transition and facilitate future maintenance and enhancements.